**Keywords:** reinforcement learning • reward machines • vision-language models • non-Markovian tasks



# RM-VLM: From Natural-Language Task Descriptions to Vision-Grounded Reward Machines

*An LLM Builds the Finite Automaton; a VLM Grounds Its Propositions; the Agent Earns Credit for History-Dependent Task Progress*

**By** Roko Parac, Christos Ziakas, Leo Ardon

arXiv:2611.08308 • cs.AI • submitted 6 Nov 2026

Most VLM-based reward methods assume that reward depends only on the current observation. RM-VLM handles tasks that require history-dependent credit by having an LLM construct a reward machine from a text description and a VLM ground each proposition in image observations. Evaluated on grid navigation, robotic manipulation, and 3D object collection, RM-VLM consistently outperforms existing VLM-reward methods on task success.

**AT A GLANCE**

**Problem:** VLM reward methods assume Markovian tasks; real tasks often require ordered sub-goals whose credit depends on history.

**Why it matters:** Without history-aware reward, RL agents fail to distinguish correct-order completions from incorrect-order ones and cannot reliably learn ordered multi-step tasks.

**Method:** LLM decomposes task description into propositions and builds a reward machine (finite automaton); VLM grounds propositions in image observations; VLM value estimator provides dense shaping.

**Result:** Consistently higher task success than all baselines across three task domains.

**Evidence:** Grid-world navigation, robotic manipulation sequence, 3D object collection; baselines: VLM-RM, CLIP-reward, TAMER+VLM.

## The Big Picture

scoring image observations against a natural-language goal description. This works well for Markovian goals where reward depends only on the current state. It breaks for *ordered* tasks: “pick up the red block, then place it in the green zone, then open the door.” A VLM reward evaluating only the current image cannot distinguish whether the red block was placed *before* or *after* the green zone interaction.

Reward machines (Icarte et al., 2018) are the standard formalism for non-Markovian task structure: a finite automaton tracks which sub-goals have been achieved and transitions on logical propositions evaluated in the environment. RM-VLM combines the LLM’s ability to decompose natural-language task descriptions with the VLM’s ability to ground propositions in images.

## Why Existing Approaches Fall Short

**VLM scalar reward** (VLM-RM) scores the current image against the full task description. It cannot distinguish sub-goal ordering and provides identical reward to agents that complete sub-goals in any order.

**CLIP-reward** embeds image and text goal in CLIP space. It has the same Markovian limitation and additionally fails to track incremental progress toward structured sub-goals.

**Handcrafted reward machines** require a domain expert to specify the propositions and automaton structure manually—the bottleneck RM-VLM eliminates.

## Core Mechanism

Given task description T, an LLM produces: a set of Boolean propositions $\{p_1, \ldots, p_n\}$ over environment entities; and a finite automaton $M = (Q, q_0, q_f, \delta, \rho)$ where $\delta: Q \times 2^P \to Q$ is the transition function and $\rho: Q \times Q \to \Re$ assigns reward to transitions. The LLM outputs M as a JSON object, which is validated and compiled.

At each step, the VLM receives image $o_t$ and each proposition $p_i$, returning $P(p_i=\text{true} \mid o_t)$. The resulting Boolean vector advances M’s state through $\delta$, triggering the reward $\rho(q_t, q_{t+1})$ on any transition.

## Technical Formulation

reward:

```
R_M(s_t, a_t) = ρ(q_t, q_{t+1}) + r^+_t
```

where $q_{t+1} = \delta(q_t, L(o_t))$ and L is the VLM labelling function with threshold $\tau = 0.5$. The shaped bonus from a VLM value estimator is:

```
r^+_t = λ ∑_{p_i ∈ pending(q_t)} P(p_i=true | o_t)
```

The augmented state $(s_t, q_t)$ is Markovian in this MDP, so standard policy-gradient methods (PPO, SAC) apply.

## Learning or Inference Procedure

(1) Submit task description to LLM to obtain M as JSON. (2) Validate M (check $q_f$ reachable, no duplicate states). (3) Initialize policy $\pi_\theta$ with PPO; use $(s_t, q_t)$ as augmented state. (4) At each step, query VLM for $L(o_t)$, advance M, and compute $R_M$. (5) Train for 5M environment steps. No fine-tuning of the LLM or VLM is performed; both are frozen.

## What the Guarantee Says

No formal guarantee is claimed. The empirical claim is consistent task-success superiority over all baselines in all three domains tested. No sample complexity bound is given.

## Experimental Findings

61%, CLIP-reward 54%, TAMER+VLM 70%.

- **Robotic manipulation sequence:** RM-VLM 74% vs. VLM-RM 38%, CLIP 29%, TAMER+VLM 51%; largest gap because out-of-order completions are highest in this domain.
- **3D object collection:** RM-VLM 81% vs. VLM-RM 58%.
- **Dense shaping:** Removing the shaped bonus reduces manipulation success from 74% to 52%.

## Ablations and Interpretation

**LLM construction errors:** On 15% of task descriptions, the LLM produces a machine with at least one missing transition; retries resolve 90% of these cases.

**VLM grounding accuracy:** Replacing the VLM with oracle labels improves manipulation success from 74% to 88%, establishing an upper bound from grounding limitations.

**Threshold τ:** $\tau = 0.5$ optimal; $\tau = 0.3$ causes false-positive transitions; $\tau = 0.7$ misses valid ones. **Shaping coefficient λ:** $\lambda = 0.1$ is optimal; larger values cause reward-shaping pathology.

## Reference

Roko Parac, Christos Ziakas, Leo Ardon. **RM-VLM: From Natural-Language Task Descriptions to Vision-Grounded Reward Machines.** arXiv:2611.08308. arxiv.org/abs/2611.08308